

ROADRESQ

System Architecture Document

Version 1.0 • Production-Oriented Architecture Baseline

Product: Digital Roadside Assistance & Automotive Service Platform

Architecture: Modular Monolith → Service-Oriented Evolution

Target stack: Next.js / React • FastAPI • PostgreSQL/PostGIS • Redis • WebSockets

CLIENTS → API/AUTH → DOMAIN MODULES → DATA + REALTIME → EXTERNAL SERVICES

Document Control

Field	Value
Document	System Architecture Document
Product	RoadResQ
Version	1.0
Status	Architecture baseline
Audience	Software architects, backend/frontend engineers, DevOps, QA, security and operations
Primary architectural style	Modular monolith with clear domain boundaries; independently scalable services later

Architecture Principles

- Server-authoritative business state: the backend is the source of truth for bookings, payments, permissions and status transitions.
- Modular boundaries: customer, provider, dispatch, booking, job, billing and support logic remain separated even when deployed together.
- API-first integration: clients communicate through versioned contracts rather than direct database access.
- Realtime where valuable: WebSockets are used for active job status and location while persistent state remains in PostgreSQL.
- Security by default: least privilege, object-level authorization, secure secrets and auditable administrative actions.
- Graceful degradation: external provider failures must not corrupt core booking state.
- Observable production behavior: logs, metrics, traces and business events are designed into the platform.

1. Architecture Overview

1.1 High-Level Architecture

Web / Mobile Clients communicate with the platform through HTTPS APIs and authenticated WebSockets. The application layer contains domain modules for identity, customers, vehicles, providers, matching, bookings, jobs, billing, notifications and administration. PostgreSQL/PostGIS stores authoritative transactional data; Redis supports caching, short-lived state and coordination; background workers process asynchronous tasks.

Logical flow:

CLIENTS → CDN/EDGE → API GATEWAY / REVERSE PROXY → FASTAPI APPLICATION → DOMAIN MODULES → POSTGRESQL/POSTGIS + REDIS → WORKERS / WEBSOCKETS → EXTERNAL INTEGRATIONS

1.2 Why a Modular Monolith

The initial production architecture should avoid unnecessary microservice complexity. A modular monolith provides transactional consistency and simpler deployment while maintaining internal boundaries. High-load domains such as realtime tracking or dispatch can later be extracted behind stable APIs/events.

2. Architecture Layers

Layer	Responsibilities	Key technologies
Client	Customer, provider and admin interfaces	Next.js, React, TypeScript, React Native/Expo
Edge	TLS termination, routing, rate limiting, static delivery	CDN / reverse proxy / WAF
API	Authentication, validation, authorization, API contracts	FastAPI, Pydantic, OpenAPI

Layer	Responsibilities	Key technologies
Domain	Business rules and state machines	Python domain modules/services
Data	Transactions, geospatial queries, persistence	PostgreSQL, PostGIS
Cache/Coordination	Short-lived cache, locks, rate limits, queues	Redis
Realtime	Active status/location events	WebSockets
Async	Notifications, matching retries, cleanup, reconciliation	Worker + queue architecture
Storage	Documents, photos and media	S3-compatible object storage
External	Maps, payments, OTP/SMS, push/email	Provider adapters
Observability	Logs, metrics, traces and alerts	OpenTelemetry-compatible stack

3. Major System Components

3.1 Client Applications

- **Customer app/web:** request assistance, manage vehicles, track provider, approve estimates, pay and review.
- **Provider app/web:** onboarding, verification, availability, request acceptance, navigation/tracking, inspection, estimates and earnings.
- **Admin portal:** verification, operations, users, bookings, payments, disputes, support and analytics.
- Clients maintain presentation state only; authoritative business state is retrieved from APIs/realtime events.

3.2 API & Application Layer

FastAPI exposes versioned HTTP APIs and authenticated WebSockets. Pydantic schemas validate input/output.

Authentication middleware establishes identity; authorization dependencies enforce role and object ownership; domain services execute business rules; repositories/adapters isolate persistence and external integrations.

Recommended backend structure:

```
app/ api/ # versioned HTTP routes auth/ # identity, sessions, RBAC customers/ # profiles and vehicles
providers/ # onboarding, verification, availability dispatch/ # matching and assignment bookings/ # booking
lifecycle jobs/ # inspection and estimates billing/ # invoices, payments, payouts realtime/ # websocket
connections/events notifications/ # push/SMS/email adapters support/ # tickets/disputes admin/ # operational
controls infrastructure/ # DB, Redis, storage, integrations workers/ # asynchronous jobs common/ # errors,
logging, config, utilities
```

4. Domain Architecture

Domain	Primary responsibility	Important dependencies
Identity	Users, sessions, roles, account status	DB, OTP provider
Customer	Profiles, vehicles, service history	Identity, Booking
Provider	Business profile, documents, services, availability	Identity, Verification
Dispatch	Eligibility, ranking, assignment, retry	Provider, Booking, Geo
Booking	Request and state machine	Customer, Provider, Dispatch
Job	Arrival, inspection, estimate, completion	Booking, Billing
Billing	Invoice, payment, payout, reconciliation	Job, payment gateway
Realtime	Status and provider location events	Booking, Redis/WebSocket
Notifications	Event-driven user communication	All relevant domains

Domain	Primary responsibility	Important dependencies
Support	Tickets, disputes and operational resolution	Users, Bookings, Billing
Admin	Platform operations and audit	All domains, Audit

5. Core Data Architecture

5.1 Primary Database

PostgreSQL is the system of record for transactional data. PostGIS provides geospatial types, indexes and queries required for provider proximity and location-aware matching.

5.2 Core Relationship

USER → VEHICLE → BOOKING → PROVIDER → JOB CARD → ESTIMATE → INVOICE → PAYMENT → REVIEW

5.3 Data Ownership

- Booking owns the authoritative lifecycle and references customer, vehicle and provider.
- Booking status history provides an audit trail of state transitions.
- Provider location updates are optimized for realtime use and retained according to privacy/operational policy.
- Financial entities remain separate from operational job data and support reconciliation.
- Provider verification documents are stored outside the relational database with metadata and controlled access in PostgreSQL.

6. Geospatial & Location Architecture

Provider discovery requires geospatial filtering and ETA estimation. PostGIS stores normalized coordinates for relevant operational locations. A mapping adapter provides geocoding, route calculation and ETA.

Location flow:

PROVIDER DEVICE → AUTHENTICATED WEBSOCKET/HTTP → LOCATION VALIDATION → REDIS/REALTIME → POSTGIS PERSISTENCE (POLICY-BASED) → CUSTOMER MAP

The system should avoid indefinite high-frequency storage of raw GPS points. Sampling, retention and precision should follow operational requirements and privacy policy.

7. Matching & Dispatch Architecture

7.1 Matching Pipeline

REQUEST → VALIDATE → FIND AVAILABLE PROVIDERS → FILTER SERVICE CAPABILITY → FILTER VEHICLE COMPATIBILITY → GEO FILTER → RANK BY ETA/QUALITY → DISPATCH → TIMEOUT → RETRY/ASSIGN

7.2 Assignment Consistency

Provider assignment must be protected using database transactions/unique constraints and, where required, distributed coordination through Redis. The backend must re-check booking eligibility immediately before final assignment.

7.3 Ranking Inputs

Signal	Purpose
Availability	Exclude providers who cannot currently accept work
Service capability	Ensure the provider can perform the requested service
Vehicle compatibility	Avoid unsuitable providers

Signal	Purpose
Distance / ETA	Prioritize faster assistance
Rating	Quality signal
Acceptance/completion performance	Operational reliability
Configurable business rules	Allow city/service-specific tuning

8. Booking State Architecture

Server-side state machine:

REQUESTED → SEARCHING → PROVIDER_ASSIGNED → ACCEPTED → ON_THE_WAY → ARRIVED → INSPECTION → ESTIMATE_PENDING → APPROVED → IN_PROGRESS → COMPLETED → INVOICE_GENERATED → PAYMENT_COMPLETED → CLOSED

Exception states: CANCELLED, EXPIRED, DISPUTED. Every transition is validated against the previous state, actor permissions and business rules. Invalid client-proposed transitions must be rejected.

9. Realtime Architecture

WebSockets provide active booking events such as provider accepted, status changes, arrival, estimate updates and location updates. Redis may be used for connection coordination, transient state and pub/sub when multiple application instances are running.

Event type	Example	Persistence
Booking status	provider.accepted	Persist status + history
Provider location	provider.location.updated	Transient/recent + policy-based history
Estimate	estimate.created / approved	Persist estimate + decision
Payment	payment.updated	Persist financial status
Notification	notification.created	Persist delivery record

10. Asynchronous Processing

- Dispatch timeout and retry jobs.
- Push/SMS/email notification delivery.
- Payment reconciliation and webhook retry handling.
- Provider/customer cleanup and expired request processing.
- Analytics/event processing.
- Document processing or verification tasks.
- Scheduled maintenance and operational reports.

Async jobs must be idempotent, retryable, observable and safe to execute more than once.

11. External Integration Architecture

Integration	Purpose	Architectural rule
Maps / Routing	Maps, geocoding, ETA, route	Use provider adapter; never spread vendor-specific logic
Payment Gateway	UPI/payment processing, webhooks	Verify callbacks; use idempotency and reconciliation
OTP/SMS	Authentication and critical notifications	Adapter + rate limits

Integration	Purpose	Architectural rule
Push Notifications	Realtime user alerts	Token management and delivery tracking
Email	Receipts/support/system mail	Asynchronous adapter
Object Storage	Photos, videos, provider documents	Private buckets + signed access

12. Security Architecture

- **Edge:** TLS, WAF/rate limiting where available, request-size limits.
- **Identity:** secure sessions/tokens, OTP controls and account status checks.
- **Authorization:** RBAC plus object-level ownership/relationship checks.
- **Data:** encryption in transit; controlled access to sensitive stored data.
- **Secrets:** environment/secret manager; never commit keys or credentials.
- **Uploads:** type/size validation, malware scanning where appropriate, private storage and signed access.
- **Payments:** do not store unnecessary sensitive payment credentials; rely on gateway tokens and verified webhooks.
- **Audit:** provider verification, account changes, disputes, payments and privileged actions are auditable.
- **Abuse controls:** rate limits, duplicate request prevention, suspicious activity monitoring.

13. Deployment Architecture

13.1 Production Topology

Internet → CDN/WAF → Load Balancer/Reverse Proxy → Next.js/Web + FastAPI Instances → PostgreSQL/PostGIS + Redis + Worker Instances → Object Storage / External APIs

13.2 Environment Separation

Environment	Purpose	Rules
Development	Local feature development	Safe test data, local services or development providers
Staging	Pre-production validation	Production-like configuration with test integrations
Production	Real users and transactions	Restricted access, monitoring, backups, change control

13.3 Containerization

Docker is recommended for reproducible application, worker and infrastructure-adjacent environments. Infrastructure services may be managed by a cloud provider where appropriate.

14. Scalability Architecture

- Keep API/application instances stateless so they can scale horizontally.
- Use Redis for shared transient state where multiple instances require coordination.
- Use PostgreSQL connection pooling and indexed queries.
- Use PostGIS indexes for proximity queries.
- Move heavy work to background workers.
- Use read replicas later for analytics/read-heavy workloads if justified.
- Extract dispatch/realtime services only when measured load or team boundaries justify it.
- Object storage and CDN should handle large media rather than application servers.

15. Reliability & Failure Architecture

Failure	Response
API instance crash	Load balancer routes traffic to healthy instances; state remains in DB/Redis.
Redis outage	Core transactional flows must degrade safely; do not lose authoritative booking/payment
DB transient failure	Retry safe operations; use connection health and transaction handling.
Map provider outage	Show limited location functionality and stale/approximate ETA where possible.
Payment gateway outage	Keep payment pending/unknown until reconciled; never mark success from timeout al
Notification provider outage	Queue/retry and expose delivery status.
WebSocket disconnect	Client reconnects and fetches authoritative current state from API.
Worker failure	Retry idempotent jobs and alert after threshold.

16. Observability Architecture

- **Logs:** structured JSON logs with request ID, user/provider/job identifiers where safe.
- **Metrics:** API latency/error rate, queue depth, active requests, match rate, provider response time, payment failures and WebSocket health.
- **Tracing:** distributed traces across API → DB/cache → external services where feasible.
- **Business monitoring:** matching rate, ETA accuracy, cancellations, completion rate and payment reconciliation.
- **Alerts:** critical error rates, database health, queue backlog, payment anomalies and availability incidents.

17. Security Boundaries & Trust Zones

Zone	Trust level	Controls
Public client	Untrusted	Validation, authentication, rate limiting
API edge	Controlled boundary	TLS, WAF/rate limits, request filtering
Application	Trusted execution	RBAC, domain rules, secure configuration
Database	Highly restricted	Private network, credentials, encryption/backups
Object storage	Restricted	Private objects, signed URLs, access policy
External services	Third-party	Adapter isolation, timeouts, retries, webhook verification
Admin operations	Privileged	Strong authentication, least privilege, audit logs

18. Performance Targets & Capacity Planning

Exact production SLOs must be validated with load testing. Initial engineering targets should include predictable API latency for common operations, reliable dispatch under concurrent requests, bounded WebSocket update frequency, and controlled database connection usage.

Capacity should be measured using realistic scenarios: concurrent assistance requests, provider location updates, dispatch fan-out, estimate approvals, payment webhooks and admin reporting.

19. Backup & Disaster Recovery

- Automated PostgreSQL backups with tested restore procedures.

- Point-in-time recovery where supported and justified.
- Separate backup storage/security boundaries from the primary database.
- Document target RPO/RTO and validate them through recovery drills.
- Store infrastructure/configuration definitions in version control.
- Maintain an incident recovery runbook for database, application, payment and external-service failures.

20. Architecture Decision Records

ADR	Decision
ADR-001	Start with a modular monolith rather than microservices.
ADR-002	Use PostgreSQL/PostGIS as the transactional and geospatial system of record.
ADR-003	Use Redis for transient coordination/caching, not as the authoritative booking database.
ADR-004	Use WebSockets for active realtime UX while persisting authoritative state.
ADR-005	Isolate external vendors behind adapters.
ADR-006	Use object storage for media/documents.
ADR-007	Use background workers for retryable/asynchronous work.

21. Architecture-to-Project Mapping

Artifact	Role in architecture
SRS v1.0	Defines what the system must do.
TRD v1.0	Defines technical direction and technology choices.
Backend Schema v1.0	Defines backend modules, endpoints, schemas and implementation contracts.
Database Schema v1.0	Defines persistence model and relationships.
App Flow v1.0	Defines primary user journeys and screen flow.
Implementation Plan v1.0	Defines build sequence and delivery milestones.
System Architecture v1.0	Defines how all components connect and operate as a production system.

22. Production Readiness Checklist

- Architecture boundaries implemented and reviewed.
- API contracts versioned and documented.
- Database migrations tested and rollback/recovery procedure defined.
- Authentication, RBAC and object-level authorization tested.
- Dispatch assignment race conditions tested.
- Realtime reconnect and stale-state behavior tested.
- Payment webhook/idempotency/reconciliation tested.
- Backups and restore tested.
- Monitoring, logs, metrics and alerts configured.
- Staging environment mirrors production architecture closely.

- Load/security/E2E tests pass critical thresholds.
- Operational runbooks and incident procedures are available.

23. Final Architecture Summary

RoadResQ should launch as a secure, observable modular monolith backed by PostgreSQL/PostGIS, Redis and authenticated WebSockets, with asynchronous workers and adapter-based integrations for maps, payments, notifications and storage. The architecture deliberately separates domain responsibilities while avoiding premature microservice complexity. As real traffic grows, dispatch, realtime tracking, notifications and analytics can be independently scaled or extracted without changing the core customer/provider contract.